Project Proposal:

Soralia Village Home-0wner's Association (HOA) - Community Directory & Resource Hub

Prepared For: Soralia Village Homeowners & Residents Association

Prepared By: Netbones Solutions Pty Ltd (registration 2020/462646/07)

admin@netbones.co.za

Date: 30 March 2025

Version: 1.0

1. Introduction

Netbones is pleased to present this proposal to design and develop a fully functional "Community Directory and Resource Hub" application for the Soralia Village Homeowners & Residents' Association. Leveraging our extensive experience in full-stack web development, we are confident in our ability to deliver a robust, user-friendly, and scalable platform that meets Soralia Village's unique needs. This application will serve as a central digital point for residents, HOA staff, and administrators, fostering community engagement, streamlining access to services, and consolidating essential knowledge.

2. Project Understanding & Vision

We understand Soralia Village requires a modern web application to:

- **Enhance Community Connection: Facilitate easier discovery of neighbours and available services within the community.**
- **Centralize Information: Provide a single source of truth for security & maintenance information, DIY guides (like the Geyser example I mentioned at the AGM), and other relevant knowledge.**
- **Improve Service Accessibility: Create a searchable directory of services offered *by* residents (e.g., artisan crafts) or *to* residents (e.g., pharmacy deliveries).**
- **Streamline Operations: Offer distinct interfaces for different user roles (Residents/Users, Staff, Administrators) with role-specific functionalities and permissions.**
- **Empower Residents: Allow residents to manage their household profiles and related service information as well as generate content.**

Our vision is to build a secure, intuitive, and maintainable platform utilizing modern web technologies (React front-end, Node.js backend, PostgreSQL database) that becomes an indispensable tool for the Soralia Village community.

3. Proposed Solution Architecture

We propose a modular, scalable architecture based on the following components:

- Frontend: A Single Page Application (SPA) built with React. This will provide a dynamic, responsive, and interactive user experience. We will utilize state management libraries (e.g Zustand) and component libraries (e.g., Material UI) for consistency and rapid development.
- Backend: A robust RESTful API server (likely built with Node.js and/or Express.js, though Python//Flask is also an option. This API will handle business logic, data validation, authentication, authorization, and communication with the database.
- Database: A NetbonesDB serverless PostgreSQL database. This provides the power and reliability of PostgreSQL with modern benefits like serverless scaling, branching for development workflows, and cost-effectiveness.
- API Communication: Secure communication between the frontend and backend via HTTPS REST functions using tRPC. JSON Web Tokens (JWT) will be used for managing user sessions and authentication.
- Mobile Client: A universal binary that can interface with our API using React Native available via Google Play Store or Apple Store.

Diagrammatic Overview (Conceptual):

```
   [ User Browser (React Frontend) ] <---- HTTPS/tRPC API ----> [ Backend API Server
(Node.js) ] <---- SQL ----> [ Netbones PostgreSQL DB ]
   |                              |
   |--- User Interface                      |--- Business Logic
   |--- Component State                     |--- Authentication & Authorization
   |--- API Calls                           |--- Data Validation
                                            |--- Database Interaction (ORM/Query Builder)
```

4. Core Features & Functionality

The application will include the following core features:

- 4.1 User Authentication & Authorization:
    - Secure user registration and login (email/password).
    - Role-based access control (RBAC) differentiating between Administrators, Staff, and standard Users (Residents).
    - Password reset functionality.
- 4.2 Role-Specific Dashboards:
    - Administrator Dashboard:
        - User Management (view, activate, deactivate, assign roles).

        - Site Configuration (e.g., defining service categories, resource categories).
        - Content Moderation Oversight (approving/rejecting user-submitted services or resources).
        - System Analytics (optional: user activity, popular resources).
        - Manage Staff Accounts.
    - Staff Dashboard:
        - Manage Official Resource Hub Content (create, edit, delete articles/guides).
        - Moderate User-Submitted Services/Resources (approve, reject, edit).
        - View Resident Directory (respecting privacy settings).
        - Potentially manage official announcements or events.
        - Assist users with profile issues.
    - User (Resident) Dashboard:
        - View/Edit Personal Household Profile.
        - Create Content, (collaborate on a Community Journal etc)
        - Manage associated Services (add services they offer or receive, e.g., link to pharmacy delivery).
        - Browse/Search the Community Directory (other residents, based on privacy settings).
        - Browse/Search the Service Directory.
        - Access the Resource Hub/Knowledge Base.
        - Control privacy settings for their profile/service visibility.
- 4.3 Individual Household/User Profiles:
    - Avatar with username + short Bio intro.
    - Basic contact information (address within Soralia, optional phone/email).
    - Ability to add household members (optional, respecting privacy).
    - Section to list/manage services linked to the household (offered or utilized).
    - Configurable privacy settings (e.g., show profile in directory? show specific services?).
- 4.4 Service Directory:
    - Categorized listing of services (e.g., Deliveries, Home Care, Artisan Crafts, Tutoring). Categories managed by Admins/Staff.
    - Services linked to specific user profiles (who offers/receives).
    - Detailed view for each service (description, contact info - respecting privacy, operating hours if applicable).
    - Search Functionality: Robust search by keywords, categories, potentially user name.
- 4.5 Resource Hub & Knowledge Base:
    - Repository for articles, guides, FAQs (e.g., "DIY Fixes for Common Geyser Issues in Soralia Homes," "Waste Management Schedule," "Association Bylaws Summary").
    - Content generated by users, managed by users, but moderated primarily by Staff/Admins.
    - Categorization and tagging for easy navigation.
    - Search Functionality: Full-text search across all resource content.
    - Potential for version history on articles.

- 4.6 Search Functionality (Global):
  - Implement efficient search across the Service Directory and Resource Hub.
  - Utilize database indexing (e.g., PostgreSQL's full-text search capabilities) for performance.

5. Technology Stack

- Frontend: React (v18+), with Next.js for SSR/SSG benefits, CSS Modules/Styled-Components/Tailwind CSS, State Management (Zustand).
- Backend: Node.js (LTS version), Express.js, ORM (e.g., Prisma) for efficient and safe database interaction.
- Database: NetbonesDB (Serverless PostgreSQL).
- Authentication: JWT (JSON Web Tokens).
- Deployment: To be discussed (options include Vercel, Netlify for frontend; AWS/GCP/Azure/Heroku/Fly.io for backend, leveraging Netbones serverless nature).
- Version Control: Git (using platforms like GitHub, GitLab, or Codeberg).

6. Development Process & Methodology

We follow an Agile (Scrum) development methodology to ensure flexibility, transparency, and iterative progress:

- Phase 1: Discovery & Design (Sprint 0)
  - Detailed requirements gathering workshop with Soralia stakeholders.
  - Finalize user stories and acceptance criteria.
  - Create wireframes and high-fidelity UI/UX mockups for key screens.
  - Define detailed database schema for NetbonesDB
  - Set up project infrastructure (repositories, NetbonesDB instance, CI/CD pipeline).
- Phase 2: Development Sprints (Iterative)
  - Develop features in 2-week sprints.
  - Each sprint includes planning, development, testing (unit, integration), and a demo/review.
  - Regular communication via [preferred channel: Slack, Teams, Email] and scheduled status meetings.
  - Focus on building core functionality first, followed by enhancements.
- Phase 3: Testing & Quality Assurance
  - Comprehensive testing: Unit tests, Integration tests, End-to-End (E2E) tests (e.g., using Cypress or Playwright).
  - User Acceptance Testing (UAT) conducted by Soralia representatives.
  - Performance testing and security vulnerability scanning.
- Phase 4: Deployment & Handover
  - Deploy the application to the production environment.
  - Provide comprehensive documentation (user manuals for each role, technical documentation).

- Conduct training sessions for Administrators and Staff.
- Phase 5: Post-Launch Support & Maintenance (Optional Retainer)
  - Offer ongoing support and maintenance packages to ensure the application remains up-to-date, secure, and functional.

7. Non-Functional Requirements

- Security: Implement industry best practices (HTTPS, input validation, output encoding, parameterized queries via ORM, dependency scanning, secure password hashing). Adherence to relevant data privacy regulations, (POPIA where applicable).
- Performance: Optimize frontend loading times (code splitting, lazy loading), efficient backend logic, and database query optimization (indexing).
- Scalability: Leverage Netbones serverless scaling for the database. Design the backend API to be stateless for horizontal scaling if needed in the future.
- Maintainability: Adhere to clean code principles, provide code comments, comprehensive documentation, and utilize a modular architecture.
- Usability: Ensure an intuitive and accessible user interface conforming to WCAG accessibility guidelines where feasible.

8. Team Expertise

Our team comprises two senior full-stack developers with proven track records in:

- Developing complex web applications using React, Node.js, and PostgreSQL.
- Designing and implementing secure, scalable backend APIs.
- Working specifically with community-focused platforms.
- Integrating with various third-party services and databases, including our own cloud-native solution, NetbonesDB
- Agile development methodologies and project management.

9. Assumptions

- Soralia Village Housing Association will provide timely feedback and necessary content (initial resource articles, category definitions, etc.).
- Hosting infrastructure and costs are separate and will be determined during the discovery phase.
- A domain like soralia.co.za or soralia.web.za can be registered via our domain register.
- Specific third-party integrations beyond the core functionality are out of scope unless explicitly discussed and included.
- Initial data migration (if any) from existing systems and/or interface with other sites like Residentia will be discussed and scoped separately.

10. Next Steps

We propose the following next steps:

1. **Meeting: Schedule a meeting to discuss this proposal, answer questions, and refine the project scope.**
2. **Discovery Workshop: Initiate Phase 1 with a detailed requirements and design workshop.**
3. **Formal Agreement: Finalize the project contract, timelines, and budget.**

We are excited about the opportunity to partner with Soralia Village HOA to build this valuable community asset. We are confident that our technical expertise and solution-oriented approach will result in a successful project delivery.

admin@netbones.co.za

https://netbones.co.za